ACTIVIDAD DEL TEMA 8 – API REST EN LARAVEL

CREA UN PDF CON LOS SIGUIENTES CONTENIDOS:

- Capturas de las peticiones y respuestas en REST Client, o la extensión que utilices
- Un breve archivo `README.md` explicando las rutas y cómo probarlas

Para probar las rutas he usado un archivo products.rest con las peticiones. Para que funcione, necesito instalar la extensión REST Client de Huachao Mao y, además, usar el comando php artisan serve.

Una vez hecho esto, simplemente hay que seguir los enlaces con CMD+ALT+R (en Mac).

Para la demostración, combino, los dos puntos en uno (Petición + Captura de la respuesta). La petición está en Courier New y la respuesta con colores.

```
###

GET http://localhost:8080/api/products

Accept: application/json
```


```
HTTP/1.1 200 OK Server: nginx/1.29.8 Content-Type: application/json Transfer-Encoding: chunked Connection: close X-Powered-By: PHP/8.3.30 Cache-Control: no-cache, private Date: Tue, 09 Jun 2026 17:38:01 GMT Access-Control-Allow-Origin: * { "success": true, "data": [ { "id": 1, "nombre": "Arturo", "descripcion": "Soy un mensaje muy guay", "precio": "12.32", "stock": 12 }, { "id": 3, "nombre": "Hola", "descripcion": "Mundo", "precio": "32.00", "stock": 1 }, { "id": 5, "nombre": "Hola mundo", "descripcion": "Esto es un producto que creo nuevo", "precio": "12.00", "stock": 2 }, { "id": 7, "nombre": "Producto nuevo", "descripcion": "Creado para la demostraci\u00f3n de la actividad 7", "precio": "100.00", "stock": 1 } ] }
```


```
###

GET http://localhost:8080/api/products/1

Accept: application/json
```


```
HTTP/1.1 200 OK Server: nginx/1.29.8 Content-Type: application/json Transfer-Encoding: chunked Connection: close X-Powered-By: PHP/8.3.30 Cache-Control: no-cache, private Date: Tue, 09 Jun 2026 17:40:13 GMT Access-Control-Allow-Origin: * { "success": true, "data": { "id": 1, "nombre": "Arturo", "descripcion": "Soy un mensaje muy guay", "precio": "12.32", "stock": 12 } }
```


```
###
```

```
POST http://localhost:8080/api/products HTTP/1.1

Accept: application/json

content-type: application/json


{

    "name": "Producto API REST",

    "description": "Descripción del producto nuevo",

    "price": "123.45",

    "stock": "2"

}
```

HTTP/1.1 201 Created Server: nginx/1.29.8 Content-Type: application/json Transfer-Encoding: chunked Connection: close X-Powered-By: PHP/8.3.30 Cache-Control: no-cache, private Date: Tue, 09 Jun 2026 17:40:30 GMT Access-Control-Allow-Origin: * { "success": true, "message": "Producto creado correctamente.", "data": { "id": 8, "nombre": "Producto API REST", "descripcion": "Descripci\u00f3n del producto nuevo", "precio": "123.45", "stock": "2" } }

```
###

PUT http://localhost:8080/api/products/5 HTTP/1.1

Accept: application/json

content-type: application/json


{

    "id": 5,

    "name": "Producto Modificada",

    "description": "Descripción de la nota modificada",

    "price": "666.66",

    "stock": 6

}
```

HTTP/1.1 200 OK Server: nginx/1.29.8 Content-Type: application/json Transfer-Encoding: chunked Connection: close X-Powered-By: PHP/8.3.30 Cache-Control: no-cache, private Date: Tue, 09 Jun 2026 17:40:56 GMT Access-Control-Allow-Origin: * { "success": true, "message": "Producto actualizado correctamente.", "data": { "id": 5, "nombre": "Producto Modificada", "descripcion": "Descripci\u00f3n de la nota modificada", "precio": "666.66", "stock": 6 } }

```
###

DELETE http://localhost:8080/api/products/5

Accept: application/json
```

HTTP/1.1 200 OK Server: nginx/1.29.8 Content-Type: application/json Transfer-Encoding: chunked Connection: close X-Powered-By: PHP/8.3.30 Cache-Control: no-cache, private Date: Tue, 09 Jun 2026 17:41:12 GMT Access-Control-Allow-Origin: * { "success": true, "message": "Producto eliminado correctamente." }

En este caso, me di cuenta de que el mensaje podría haber sido 204, así que lo cambié en el método del destroy y volví a probar con un objeto existente:

HTTP/1.1 204 No Content Server: nginx/1.29.8 Connection: close X-Powered-By: PHP/8.3.30 Cache-Control: no-cache, private Date: Tue, 09 Jun 2026 17:46:53 GMT Access-Control-Allow-Origin: *

AÑADE AL PDF EL CÓDIGO DE LA MIGRACIÓN, EL MODELO, EL CONTROLADOR, LA CLASE RESOURCE Y LA CLASE REQUEST

La migración (de la tabla de la base de datos) y el modelo son las mismas que en las prácticas anteriores.

Rutas API:

```php
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

use App\Http\Controllers\Api\ProductController;

Route::apiResource('products', ProductController::class)->only([
'index', 'show', 'store', 'update', 'destroy'
]
);
```

Controlador API:

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Requests\StoreProductRequest;
use App\Http\Resources\ProductResource;
use Illuminate\Http\Request;
use App\Models\Product;
use Illuminate\Http\JsonResponse;

class ProductController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index(): JsonResponse
    {
        //
        return response()->json([
            'success' => true,
            'data' => ProductResource::collection(Product::all())
        ], 200);
```

```
    }
    /**
     * Display the specified resource.
     */
    public function show(Product $product): JsonResponse
    {
        return response()->json([
            'success' => true,
            'data' => new ProductResource($product)
        ], 200);
    }
    /**
     * Store a newly created resource in storage.
     */
    public function store(StoreProductRequest $request): JsonResponse
    {

        $product = Product::create($request->validated());
        return response()->json([
            'success' => true,
            'message' => 'Producto creado correctamente.',
            'data' => new ProductResource($product)
        ], 201);
    }




    /**
     * Update the specified resource in storage.
     */
    public function update(StoreProductRequest $request, Product $product):
JsonResponse
    {
        //
        $product->update($request->validated());
        return response()->json([
            'success' => true,
            'message' => 'Producto actualizado correctamente.',
            'data' => new ProductResource($product),
        ], 200);
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(Product $product): JsonResponse
    {
        //
        $product->delete();
        return response()->json([
```

```
            'success' => true,
            'message' => 'Producto eliminado correctamente.'
        ], 204);
    }
}
```

ProductResource:

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class ProductResource extends JsonResource
{
    /**
     * Transform the resource into an array.
     *
     * @return array<string, mixed>
     */
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'nombre' => $this->name,
            'descripcion' => $this->description,
            'precio' => $this->price,
            'stock' => $this->stock
        ];
    }
}
```

Modelo:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    //
    protected $fillable = ['name', 'description', 'price', 'stock'];
    protected $guarded = ['id'];
```

```
}
```

Migración:

Al instalar las API, se me añadió esta migración para usuarios:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('personal_access_tokens', function (Blueprint $table) {
            $table->id();
            $table->morphs('tokenable');
            $table->text('name');
            $table->string('token', 64)->unique();
            $table->text('abilities')->nullable();
            $table->timestamp('last_used_at')->nullable();
            $table->timestamp('expires_at')->nullable()->index();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('personal_access_tokens');
    }
};
```

Y la migración original de la tabla:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
```

```
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('name', 255)->notNullable();
            $table->text('description');
            $table->decimal('price', 8, 2)->notNullable();
            $table->integer('stock')->notNullable();

            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

Finalmente, el StoreProductRequest actualizado para la validación API:

```
<?php

namespace App\Http\Requests;

use Illuminate\Contracts\Validation\ValidationRule;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\ValidationException;
use Illuminate\Contracts\Validation\Validator;
use Illuminate\Http\Exceptions\HttpResponseException;

class StoreProductRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
```

```
     * @return array<string, ValidationRule|array<mixed>|string>
     */
    public function rules(): array
    {
        return [
            'name' => 'required|string|min:3|max:100',
            'description' => 'required|string|min:10',
            'price' => 'required|numeric|min:0.01',
            'stock' => 'required|integer|min:0'
        ];
    }
    protected function failedValidation(Validator $validator)
    {
        throw new HttpResponseException(response()->json([
            'success' => false,
            'message' => 'Error de validación',
            'errors' => $validator->errors()
        ], 422, [], JSON_UNESCAPED_UNICODE));
    }
}
```